

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

DIVISION OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

LABORATORY RECORD

Academic Year 2026-2027

ODD SEMESTER

Name : JANE HANNA

Register Number : URK23CS1027

Course Code : 24CS2016

Course Name : BUILDING APPLICATIONS USING FLUTTER

NOVEMBER 2026



Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

MoE, UGC & AICTE Approved

NAAC A++ Accredited

DIVISION OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

LABORATORY RECORD

Academic Year 2026-2027

Course Code: 24CS2016

Course Name: BUILDING APPLICATIONS USING FLUTTER

Register No.: URK23CS1027

It is hereby certified that this is the bonafide record of work done by **Ms. JANE HANNA** during the Odd semester of the academic year 2026-2027 and submitted for the University Practical Examination held on **06.11.26**.

Faculty-in-charge Signature

Faculty-in-charge Name

Table of Contents

#	Date	Name of the Exercise	Marks	Signature
1.	24.08.26	WORKING WITH WIDGETS AND LAYOUTS		
2.	31.08.26	ADAPTIVE AND RESPONSIVE UI DESIGN		
3.	07.09.26	ADD INTERACTIVITY TO APP – INPUT AND FORMS		
4.	21.09.26	UTILIZE THE ASSETS AND MEDIA		
5.	28.09.26	IMPLEMENTATION OF NAVIGATION AND ROUTING		
6.	05.10.26	INTEGRATE ANIMATIONS AND TRANSITION		
7.	12.10.26	APPLICATION STATE MANAGEMENT AND SERIALIZATION		
8.	12.10.26	DATA PERSISTENCE USING SQLITE AND FIREBASE		
9.	12.10.26	ACCESSING APIs THROUGH HTTP		
10.	02.11.26	ADD ANDROID AS A BUILD TARGET		
11.	02.11.26	ADD iOS AS A BUILD TARGET		
12.	02.11.26	BUILD WEB APP WITH FLUTTER		

Faculty-in-charge Signature

EX - 1	WORKING WITH WIDGETS AND LAYOUTS
DATE	24.08.26

Aim: To develop a Flutter application using basic widgets and layout widgets such as Container, Row, Column, Expanded, Card, and ListView.

Description

Flutter is an open-source framework used to develop modern and cross-platform applications. It uses the Dart programming language to create efficient and interactive applications. Flutter provides a wide range of reusable widgets for designing attractive user interfaces. It supports responsive layouts that can run on different platforms from a single codebase. The framework also provides features such as hot reload for faster development and testing. Overall, Flutter makes application development simple, flexible, and efficient for developers.

Procedure

1. Check the Flutter installation using flutter doctor.
2. Check the available devices using flutter devices.
3. Create a new Flutter project using flutter create widget_layout.
4. Open the project in VS Code using code ..
5. Run the default application using flutter run.
6. Open lib/main.dart and create the basic Flutter application.
7. Add Column and Container widgets to display student information.
8. Add Row and Card widgets with icons and subject details.
9. Add a ListView to display multiple subjects or items.
10. Run the application using flutter run and observe the student profile/dashboard interface.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';


void main() {
  runApp(const MyApp());
}


class MyApp extends StatelessWidget {
  const MyApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
        appBar: AppBar(
          title: const Text('Student Profile'),
          backgroundColor: Colors.blue,
        ),
        body: SingleChildScrollView(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
```

```
// Step 6: Container for Profile Heading
Container(
  padding: const EdgeInsets.all(20),
  margin: const EdgeInsets.all(10),
  color: Colors.blue.withOpacity(0.1),
  child: const Text(
    'Student Profile',
    style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
    textAlign:
        TextAlign.center, // FIX 1: Corrected alignment syntax
  ),
),

// Step 5: Column for Student Details
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16.0),
  // FIX 2: Removed conflicting 'const' modifier from the Column structure
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: const [
      Text(
        'Student Name: Jane Hanna [URK23CS1027]',
```

```
        style: TextStyle(fontSize: 16),
      ),
      Text(
        'Department: Computer Science and Engineering (CSE)',
        style: TextStyle(fontSize: 16),
      ),
      Text(
        'Lab: Flutter Laboratory',
        style: TextStyle(fontSize: 16),
      ),
    ],
  ),
),

const SizedBox(height: 15),

// Step 7: Row with evenly spaced icons
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: const [
    Icon(Icons.home, size: 30, color: Colors.blue),
    Icon(Icons.person, size: 30, color: Colors.blue),
```

```
    Icon(Icons.settings, size: 30, color: Colors.blue),
  ],
),

const SizedBox(height: 15),

// Step 8: Individual Card Widget
Card(
  margin: const EdgeInsets.all(10),
  child: const ListTile(
    leading: Icon(Icons.book),
    title: Text('Flutter'),
    subtitle: Text('Mobile Application Development'),
  ),
),

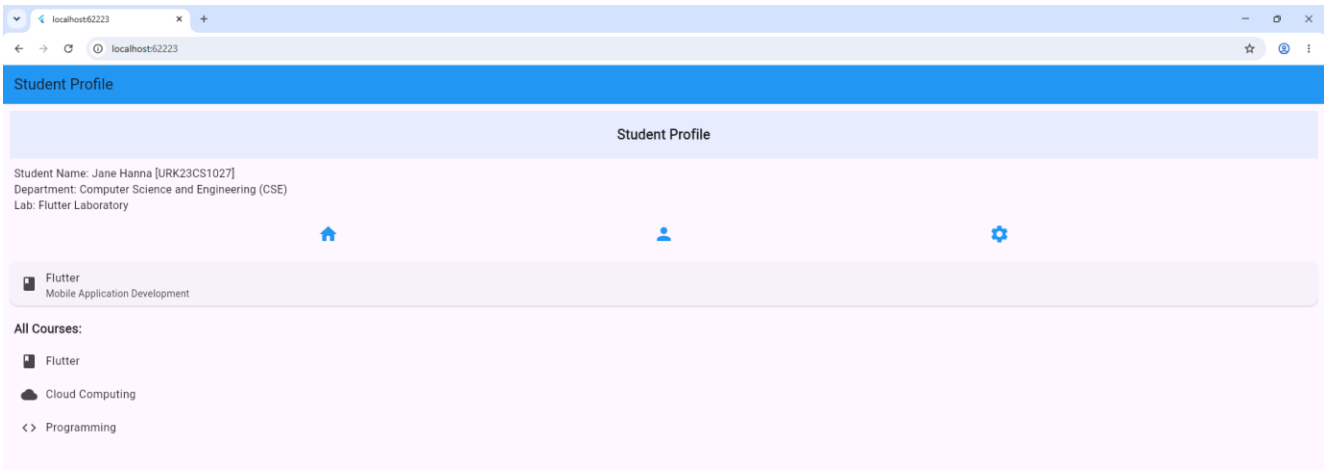
const Padding(
  padding: EdgeInsets.only(left: 16.0, top: 10),
  child: Text(
    'All Courses:',
    style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
  ),
```

```
        ),

        // Step 9: Multi-item list layout
        ListView(
          shrinkWrap: true,
          physics: const NeverScrollableScrollPhysics(),
          padding: const EdgeInsets.all(10),
          children: const [
            ListTile(leading: Icon(Icons.book), title: Text('Flutter')),
            ListTile(
              leading: Icon(Icons.cloud),
              title: Text('Cloud Computing'),
            ),
            ListTile(
              leading: Icon(Icons.code),
              title: Text('Programming'),
            ),
          ],
        ),
      ],
    ),
  ),
```

```
      ),
    );
  }
}
```

Output



Result: Thus, a Flutter student profile/dashboard application using basic widgets and layouts was successfully developed and executed.

EX - 2	ADAPTIVE AND RESPONSIVE UI DESIGN
DATE	31.08.26

Aim: To create a Flutter application that automatically adapts its layout according to the available screen size.

Description

Responsive UI design allows a Flutter application to adjust its layout based on the screen size. Flutter provides widgets such as LayoutBuilder, Column, Row, Card, and Expanded to create flexible interfaces. A smaller screen can display content vertically, while a larger screen can arrange the same content horizontally. This helps create applications that provide a suitable user interface across different devices and screen sizes.

Procedure

1. Create a new Flutter project using flutter create responsive_ui.
2. Navigate to the project folder using cd responsive_ui.
3. Open the project in VS Code using code ..
4. Run the application using flutter run.
5. Open the lib/main.dart file.
6. Use the LayoutBuilder widget to check the available screen width.
7. Create a mobile layout using a Column with Student and Courses cards.
8. Create a desktop/tablet layout using a Row with Expanded Student and Courses cards.
9. Resize the browser window or use different emulator sizes to test the application.
10. Observe how the layout changes between smaller and larger screens.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

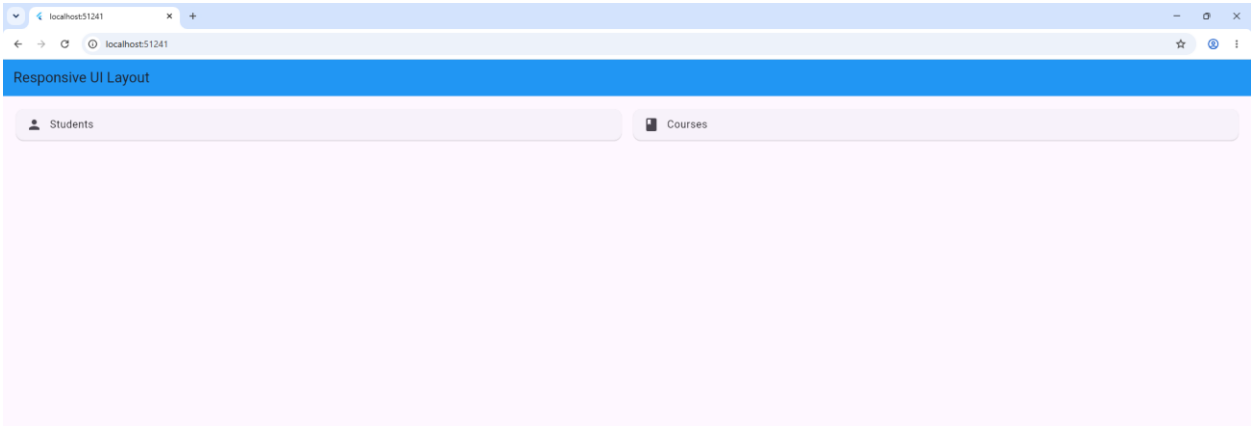
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Responsive UI Layout'),
          backgroundColor: Colors.blue,
        ),
        // Step 4: LayoutBuilder listens to changes in screen width
        body: LayoutBuilder(
          builder: (context, constraints) {
            // If the screen width is smaller than 600px, show Mobile Layout
            if (constraints.maxWidth < 600) {
              return Padding(
                padding: const EdgeInsets.all(16.0),
                // Step 5: Mobile Layout displays items vertically using a Column
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: const [
                    Card(
                      child: ListTile(
```

```
            leading: Icon(Icons.person),
            title: Text('Student'),
          ),
        ),
        Card(
          child: ListTile(
            leading: Icon(Icons.book),
            title: Text('Courses'),
          ),
        ),
      ],
    ),
  );
} else {
  // If the screen width is 600px or larger, show Desktop/Tablet Layout
  return Padding(
    padding: const EdgeInsets.all(16.0),
    // Step 6: Desktop Layout displays items horizontally using a Row
    child: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: const [
        Expanded(
          child: Card(
            child: ListTile(
              leading: Icon(Icons.person),
              title: Text('Students'),
            ),
          ),
        ),
        SizedBox(width: 10), // Adds clean spacing between horizontal cards
        Expanded(
```

```
                child: Card(
                  child: ListTile(
                    leading: Icon(Icons.book),
                    title: Text('Courses'),
                  ),
                ),
              ),
            ],
          ),
        );
      }
    },
  ),
 ),
);
}
}
```

Output



Result: Thus, a responsive Flutter application was successfully developed that automatically adjusts its layout according to the available screen size.

EX - 3	ADD INTERACTIVITY TO APP – INPUT AND FORMS
DATE	07.09.26

Aim: To create an interactive Flutter form with input fields and validation.

Description

Flutter forms are used to collect user information through interactive input fields. The Form widget provides a structure for managing form data and validation, while TextFormField is used to accept inputs such as name, email, and password. Validation helps ensure that required information is entered correctly before submitting the form. An ElevatedButton can be used to submit the form and display a confirmation message after successful validation.

Procedure

1. Create a new Flutter project using flutter create student_form.
2. Navigate to the project folder using cd student_form.
3. Open the project in VS Code using code ..
4. Open the lib/main.dart file.
5. Create a GlobalKey<FormState> to manage the form.
6. Add a TextFormField for entering the student's name with validation.
7. Add an email input field using TextFormField and set the email keyboard type.
8. Add a password field using TextFormField with obscureText.
9. Add an ElevatedButton to submit the form after validation.
10. Test the form by submitting it without entering information and observe the validation messages.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: const StudentFormScreen(),
    );
  }
}

// Creating a Stateful widget since forms track active user inputs dynamically
class StudentFormScreen extends StatefulWidget {
  const StudentFormScreen({super.key});

  @override
  State<StudentFormScreen> createState() => _StudentFormScreenState();
}

class _StudentFormScreenState extends State<StudentFormScreen> {
  // Step 3: Create global form state tracker key
  final _formKey = GlobalKey<FormState>();

  @override
```

```
Widget build(BuildContext context) {
 return Scaffold(
  appBar: AppBar(
   title: const Text('Student Registration Form'),
   backgroundColor: Colors.blue,
  ),
  body: Padding(
   padding: const EdgeInsets.all(16.0),
   child: Form(
    key: _formKey, // Attaching the validator tracker key to the framework structure
    child: ListView(
     children: [
      // Step 4: Add Student Name field with presence validation
      TextFormField(
       decoration: const InputDecoration(
        labelText: 'Student Name',
        border: OutlineInputBorder(), // Added for clean look
       ),
       validator: (value) {
        if (value == null || value.isEmpty) {
         return 'Enter student name';
        }
        return null;
       },
      ),
      const SizedBox(height: 15),

      // Step 5: Add Email input field
      TextFormField(
       decoration: const InputDecoration(
        labelText: 'Email',
```

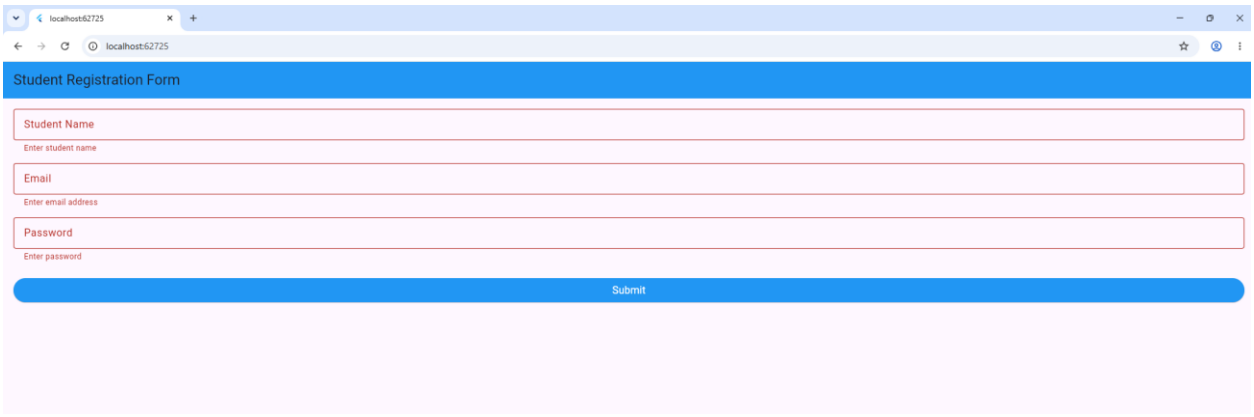
```
    border: OutlineInputBorder(),
   ),
   keyboardType: TextInputType.emailAddress,
   validator: (value) {
    if (value == null || value.isEmpty) {
     return 'Enter email address';
    }
    if (!value.contains('@')) {
     return 'Enter a valid email layout';
    }
    return null;
   },
  ),
  const SizedBox(height: 15),

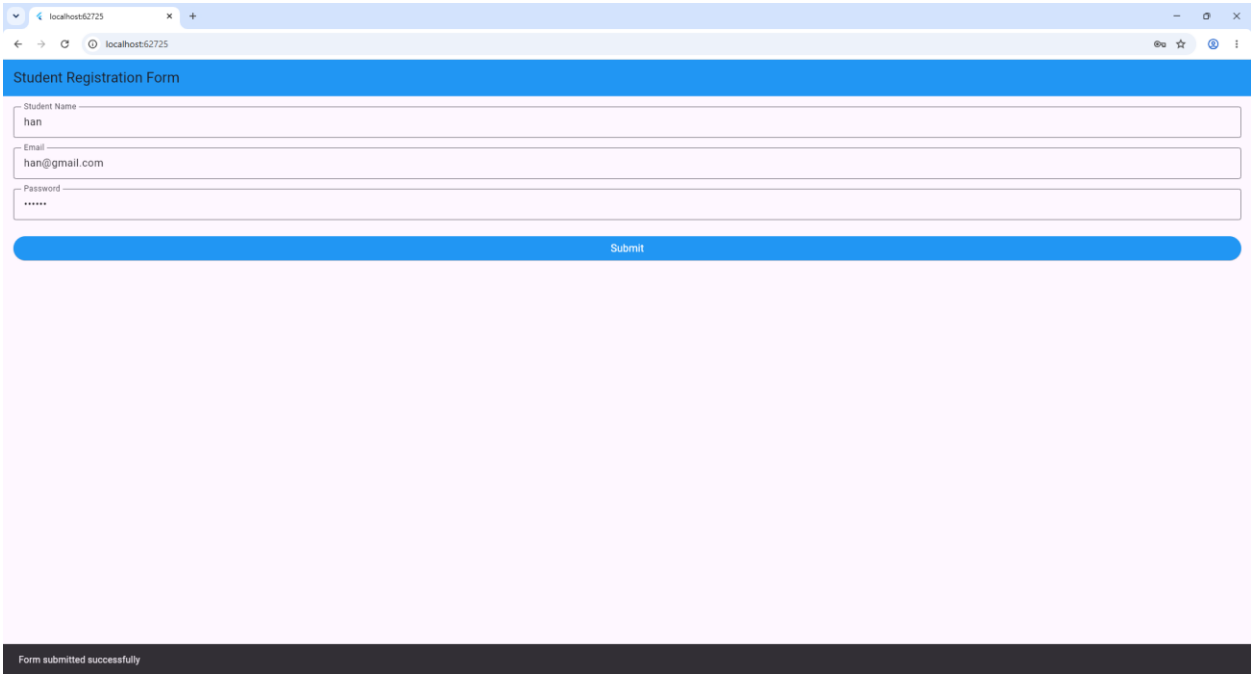
  // Step 6: Add Password hidden character element
  TextFormField(
   obscureText: true,
   decoration: const InputDecoration(
    labelText: 'Password',
    border: OutlineInputBorder(),
   ),
   validator: (value) {
    if (value == null || value.isEmpty) {
     return 'Enter password';
    }
    if (value.length < 6) {
     return 'Password must be at least 6 characters long';
    }
    return null;
   },
```

```
            ),
            const SizedBox(height: 25),

            // Step 7: Add Submit submission action button
            ElevatedButton(
              style: ElevatedButton.styleFrom(
                padding: const EdgeInsets.symmetric(vertical: 15),
                backgroundColor: Colors.blue,
                foregroundColor: Colors.white,
              ),
              onPressed: () {
                // Execute evaluation protocols across structural fields
                if (_formKey.currentState!.validate()) {
                  ScaffoldMessenger.of(context).showSnackBar(
                    const SnackBar(
                      content: Text('Form submitted successfully'),
                    ),
                  );
                }
              },
              child: const Text('Submit', style: TextStyle(fontSize: 16)),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

Output





Result: Thus, a student registration form with name, email, password, validation, and a submit button was successfully created and tested.

EX - 4	UTILIZE THE ASSETS AND MEDIA
DATE	21.09.26

Aim: To add and display image, audio, and video assets in a Flutter application.

Description

Flutter allows developers to include different types of media assets in an application. Images, audio, and video files can be organized inside an assets directory and configured through the pubspec.yaml file. The Image.asset() widget can be used to display images, while layouts such as GridView can be used to create an image gallery. Proper asset configuration makes it possible to load and display media efficiently within the application.

Procedure

1. Create a new Flutter project using flutter create media_app.
2. Navigate to the project directory using cd media_app.
3. Create an assets directory with separate folders for images, audio, and video.
4. Copy an image file into the assets/images/ folder.
5. Add the image asset path to the pubspec.yaml file.
6. Ensure the YAML indentation is correct while configuring the asset.
7. Run flutter pub get to update the project packages.
8. Use Image.asset() to load and display the configured image.
9. Create an image gallery using the GridView.count widget.
10. Run the application using flutter run and verify that the media assets are displayed successfully.

Program

[pubspec.yaml]

```
flutter:
  uses-material-design: true

  # Step 4: Asset folder configurations
  assets:
    - assets/images/
```

[lib/main.dart]

```
import 'package:flutter/material.dart';

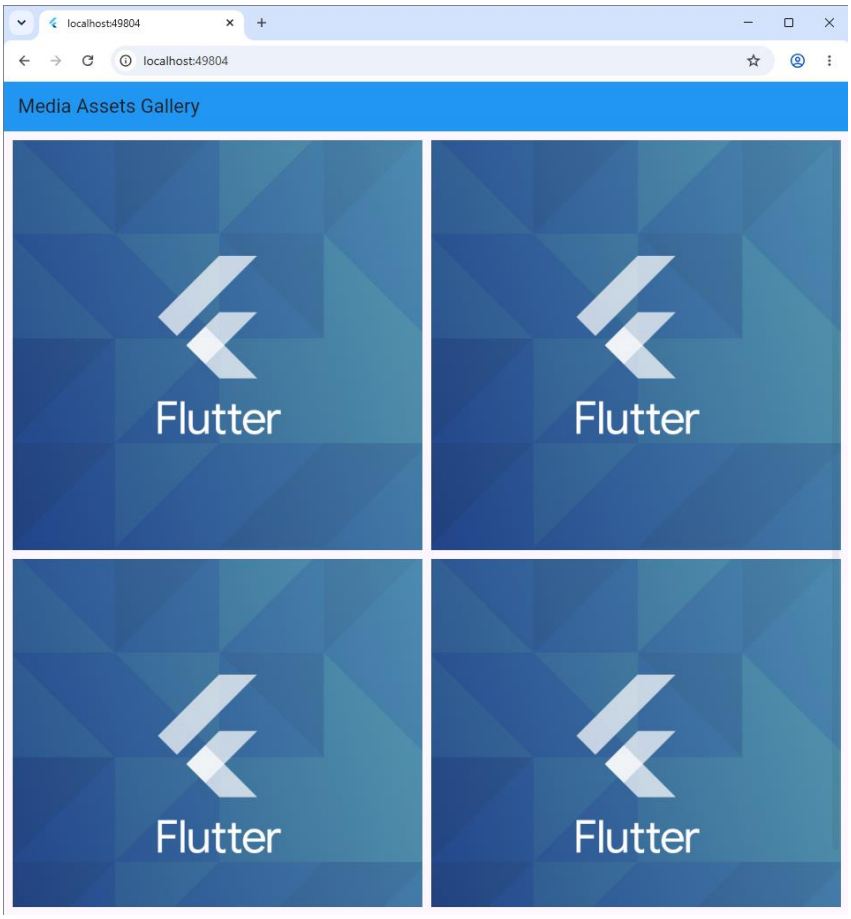
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Media Assets Gallery'),
          backgroundColor: Colors.blue,
        ),
        body: Padding(
```

```
        padding: const EdgeInsets.all(10.0),
        // Step 7: GridView counts rows and displays image modules side-by-side
        child: GridView.count(
          crossAxisCount: 2,          // Sets layout to 2 columns
          crossAxisSpacing: 10,       // Horizontal spacing between images
          mainAxisSpacing: 10,        // Vertical spacing between images
          children: [
            // Step 6: Individual image collection allocations
            Image.asset('assets/images/flutter.png', fit: BoxFit.cover),
            Image.asset('assets/images/flutter.png', fit: BoxFit.cover),
            Image.asset('assets/images/flutter.png', fit: BoxFit.cover),
            Image.asset('assets/images/flutter.png', fit: BoxFit.cover),
          ],
        ),
      ),
    ),
  );
 }
}
```

Output



Result: Thus, the Flutter application successfully loaded and displayed the configured media assets.

EX - 5	IMPLEMENTATION OF NAVIGATION AND ROUTING
DATE	28.09.26

Aim: To implement navigation between multiple Flutter screens.

Description

Navigation and routing in Flutter are used to move between different screens of an application. Flutter provides the Navigator class to push a new screen and return to the previous screen. Named routes can also be defined inside MaterialApp to make navigation between multiple screens easier and organized. This allows users to move from the Home screen to Profile and Details screens and return using back navigation.

Procedure

1. Create a new Flutter project using flutter create navigation_app.
2. Navigate to the project directory using cd navigation_app.
3. Open the project in VS Code using code ..
4. Create three screens: Home, Profile, and Details.
5. Add a Go to Profile button on the Home screen.
6. Use Navigator.push() to navigate from Home to Profile.
7. Add a View Details button on the Profile screen.
8. Use Navigator.push() to navigate from Profile to Details.
9. Use Navigator.pop(context) to return to the previous screen.
10. Define and use named routes for Home, Profile, and Details screens.

Program

[lib/home.dart]

```
import 'package:flutter/material.dart';
import 'profile.dart';

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Home Screen'),
        backgroundColor: Colors.blue,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('Welcome to Home Screen', style: TextStyle(fontSize: 18)),
            const SizedBox(height: 20),
            // Step 3: Button using standard explicit routing transition
            ElevatedButton(
              onPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => const ProfilePage(),
                  ),
                );
              },
              child: const Text('Go to Profile'),
            ),
```

```
          const SizedBox(height: 10),
          // Step 7: Alternative route verification using the Named Route framework lookup
          ElevatedButton(
            style: ElevatedButton.styleFrom(backgroundColor: Colors.grey[300]),
            onPressed: () {
              Navigator.pushNamed(context, '/profile');
            },
            child: const Text('Go to Profile (Via Named Route)'),
          ),
        ],
      ),
    ),
  );
 }
}
```

[lib/profile.dart]

```
import 'package:flutter/material.dart';
import 'details.dart';

class ProfilePage extends StatelessWidget {
 const ProfilePage({super.key});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: const Text('Profile Screen'),
    backgroundColor: Colors.green,
   ),
```

```
  body: Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Text('Welcome to Profile Screen', style: TextStyle(fontSize: 18)),
        const SizedBox(height: 20),
        // Step 4: Add explicit transition target linking deep metadata parameters
        ElevatedButton(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const DetailsPage(),
              ),
            );
          },
          child: const Text('View Details'),
        ),
        const SizedBox(height: 15),
        // Step 5: Pop back to previous page
        TextButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('← Go Back'),
        ),
      ],
    ),
  ),
);
```

```
  }
}
```

[lib/details.dart]

```
import 'package:flutter/material.dart';

class DetailsPage extends StatelessWidget {
  const DetailsPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Details Screen'),
        backgroundColor: Colors.orange,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('Detailed Information Panel', style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
            const SizedBox(height: 20),
            // Step 5: Return to previous screen
            ElevatedButton(
              style: ElevatedButton.styleFrom(backgroundColor: Colors.orange),
              onPressed: () {
                Navigator.pop(context);
              },
              child: const Text('Back to Profile'),
```

```
            ),
          ],
        ),
      ),
    );
  }
}
```

[lib/main.dart]

```
import 'package:flutter/material.dart';
import 'home.dart';
import 'profile.dart';
import 'details.dart';

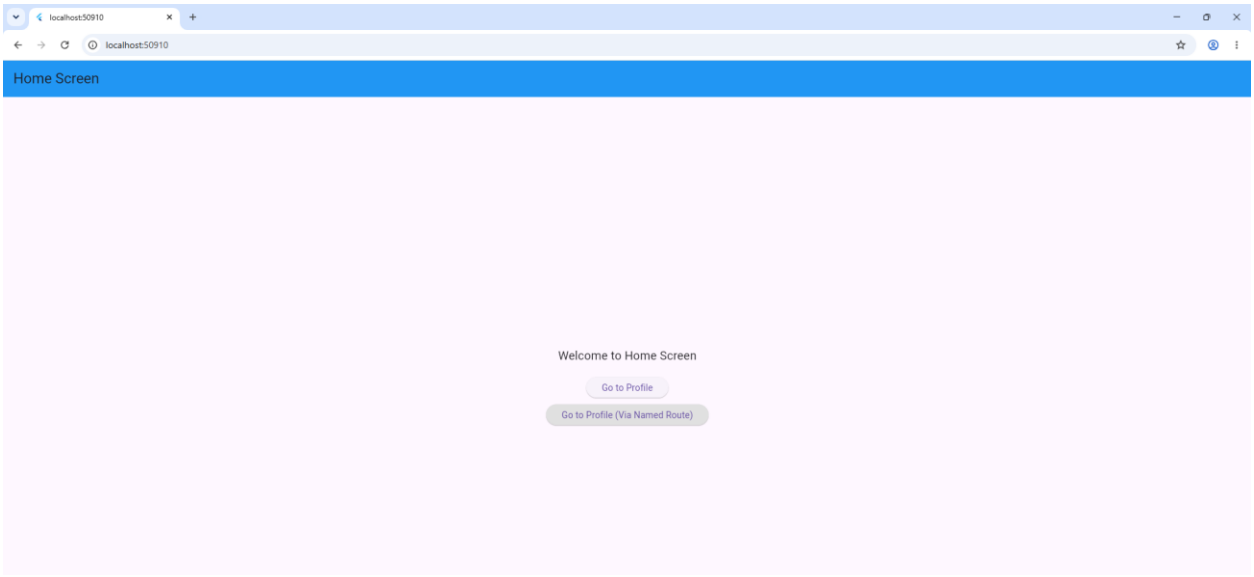
void main() {
  runApp(const MyApp());
}

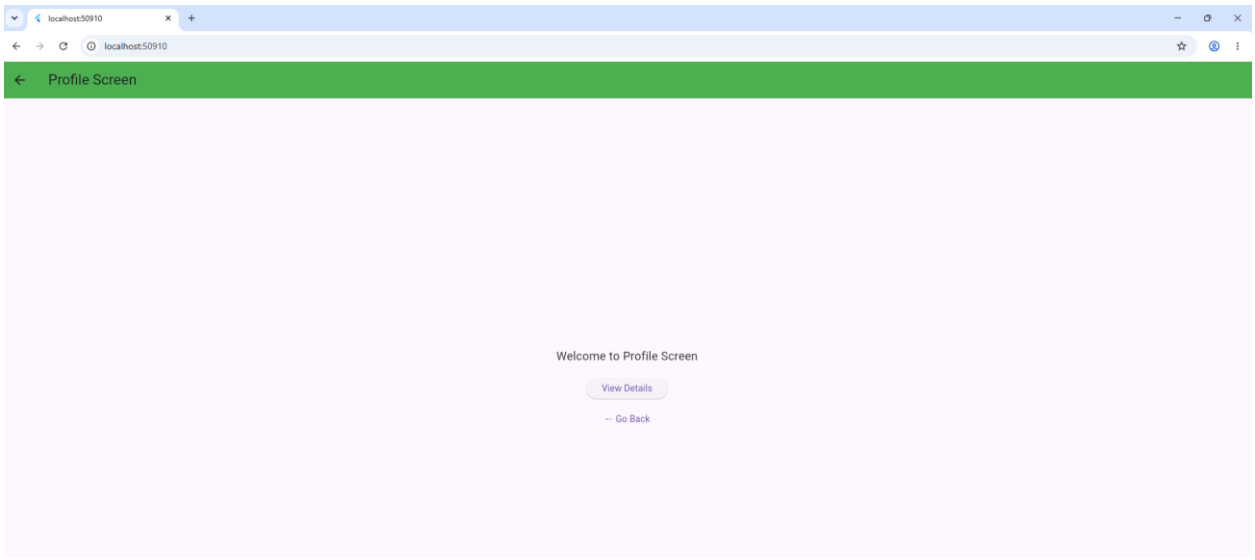
class MyApp extends StatelessWidget {
  const MyApp({super.key});

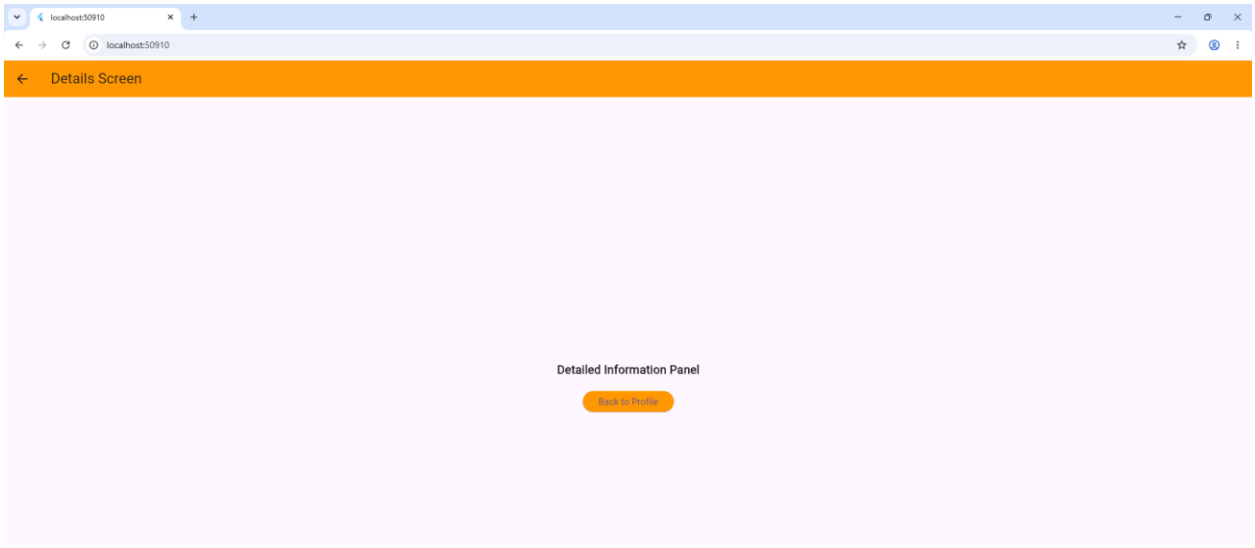
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      // Step 6: Define centralized Named Routes table structures
      initialRoute: '/',
      routes: {
        '/': (context) => const HomePage(),
        '/profile': (context) => const ProfilePage(),
```

```
        '/details': (context) => const DetailsPage(),
      },
    );
  }
}
```

Output







Result: Thus, navigation between Home, Profile, and Details screens was successfully implemented with back navigation support.

EX - 6	INTEGRATE ANIMATIONS AND TRANSITION
DATE	05.10.26

Aim: To implement animations and page transitions in Flutter using the AnimatedContainer widget and state changes.

Procedure

1. Create a new Flutter project named animation_app using VS Code and open it in the editor.
2. Open the lib/main.dart file and create the basic Flutter application structure.
3. Create an AnimatedContainer widget with a specified width, height, duration, and rounded corners.
4. Set the animation duration to one second using Duration(seconds: 1).
5. Declare a Boolean state variable isLarge = false to control the size of the animated widget.
6. Set the container's width and height using the conditional expression isLarge ? 300 : 150 to change its dimensions dynamically.
7. Add an ElevatedButton with the label Animate below the container.
8. Use the button's onPressed callback and setState() to toggle the value of isLarge whenever the button is clicked.
9. Run the application in Chrome and click the Animate button to observe the container smoothly change its size.
10. Test the animation by clicking the button repeatedly and observe the transition between the smaller and larger container sizes.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Animation App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const AnimationScreen(),
    );
  }
}

class AnimationScreen extends StatefulWidget {
  const AnimationScreen({super.key});

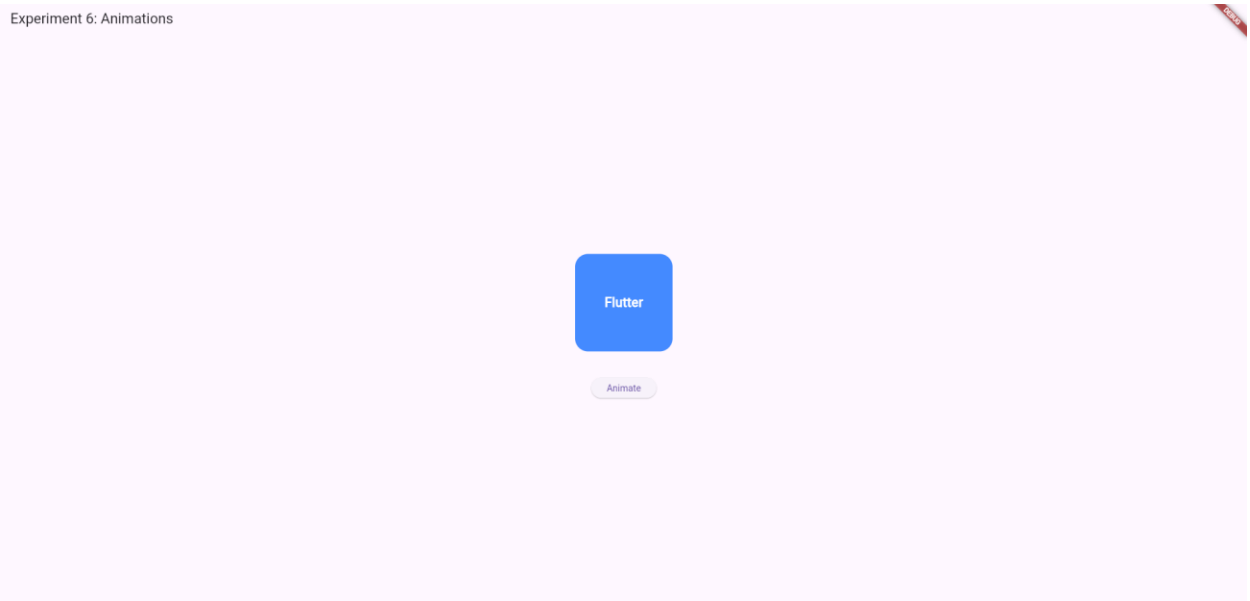
  @override
  State<AnimationScreen> createState() => _AnimationScreenState();
}

class _AnimationScreenState extends State<AnimationScreen> {
  // Step 3 – Create state variable to track size toggle
  bool isLarge = false;
```

```
@override
Widget build(BuildContext context) {
 return Scaffold(
  appBar: AppBar(
   title: const Text('Experiment 6: Animations'),
  ),
  body: Center(
   child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
     // Step 2 & 4 – Create and configure the AnimatedContainer
     AnimatedContainer(
      duration: const Duration(seconds: 1),
      width: isLarge ? 300 : 150,
      height: isLarge ? 300 : 150,
      decoration: BoxDecoration(
       color: Colors.blueAccent, // Added color to make the shape visible
       borderRadius: BorderRadius.circular(20),
      ),
      child: const Center(
       child: Text(
       'Flutter',
        style: TextStyle(color: Colors.white, fontSize: 20, fontWeight: FontWeight.bold),
       ),
      ),
     ),
     const SizedBox(height: 40), // Generates spacing between shape and button
     // Step 5 – Add the toggle trigger button
     ElevatedButton(
      onPressed: () {
```

```
            setState(() {
              isLarge = !isLarge; // Swaps the boolean value true <-> false
            });
          },
          child: const Text('Animate'),
        ),
      ],
    ),
  ),
 );
 }
}
```

Output





Result: The Flutter application was implemented successfully using the AnimatedContainer widget. The container smoothly changes its size when the Animate button is clicked, demonstrating basic animation behavior in Flutter.

EX - 7	APPLICATION STATE MANAGEMENT AND SERIALIZATION
DATE	12.10.26

Aim: To manage application state in Flutter and serialize and deserialize student data using JSON.

Procedure

1. Create a new Flutter project named state_app using the terminal commands flutter create state_app, cd state_app, and code ..
2. Create a Dart file named student.dart inside the lib folder to define the Student model.
3. Define the Student class with three properties: id, name, and course.
4. Create a constructor for the Student class using required parameters to initialize student details.
5. Implement the toJson() method to convert the student object into a Map<String, dynamic> containing the student's ID, name, and course.
6. Implement the Student.fromJson() factory constructor to convert JSON-style map data back into a Student object.
7. Create a student object with sample details, such as ID 1, name Raj, and course CSE.
8. Convert the student object into JSON-style map data using student.toJson() and display the result using print().
9. Deserialize the map data using Student.fromJson(data) and display the student's name to verify the conversion.
10. Create a list of student names, update it using setState(), and display the names dynamically using ListView.builder and ListTile.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';

import 'student.dart'; // Imports the student model we created in Step 2

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'State & Serialization App',
      theme: ThemeData(primarySwatch: Colors.teal),
      home: const StudentStateScreen(),
    );
  }
```

```
}

class StudentStateScreen extends StatefulWidget {
  const StudentStateScreen({super.key});

  @override
  State<StudentStateScreen> createState() => _StudentStateScreenState();
}

class _StudentStateScreenState extends State<StudentStateScreen> {
  // Step 6 – Main app state array holding student names for the UI list
  List<String> students = [];

  // Terminal log storage to print serialization steps clearly on screen
  String logs = "Logs will appear here when you add a student.";

  void _addAndSerializeStudent() {
    // Step 3 – Create sample student object instance
    Student student = Student(
      id: students.length + 1,
      name: 'Jane ${students.length + 1}',
      course: 'CSE',
```

```
    );

    // Step 4 – Convert object to JSON map structure
    Map<String, dynamic> data = student.toJson();

    // Step 5 – Convert JSON map back to a fresh object instance
    Student newStudent = Student.fromJson(data);

    // Update screen state with new logs and append name to the visible list
    setState(() {
      logs =
          "1. Object Created: ${student.name} (${student.course})\n"
          "2. Serialized Map Data: $data\n"
          "3. Deserialized Object Back Name: ${newStudent.name}";

      students.add(newStudent.name); // Appends student name to string array
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```
appBar: AppBar(title: const Text('Experiment 7: State & JSON')),
body: Padding(
  padding: const EdgeInsets.all(16.0),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      // Console-like visualization block for data serialization tracking
      Card(
        color: Colors.grey[200],
        child: Padding(
          padding: const EdgeInsets.all(12.0),
          child: Text(
            logs,
            style: const TextStyle(fontFamily: 'monospace', fontSize: 13),
          ),
        ),
      ),
      const SizedBox(height: 16),

      // Step 7 – Dynamic ListView builder showing registered students
      Expanded(
        child: students.isEmpty
```

```
            ? const Center(child: Text('No students added yet.'))
            : ListView.builder(
                itemCount: students.length,
                itemBuilder: (context, index) {
                  return Card(
                    margin: const EdgeInsets.symmetric(vertical: 4),
                    child: ListTile(
                      leading: CircleAvatar(child: Text('${index + 1}')),
                      title: Text(students[index]),
                      subtitle: const Text('Course: CSE'),
                    ),
                  );
                },
              ),
        ),
      ],
    ),
  ),
  floatingActionButton: FloatingActionButton.extended(
    onPressed: _addAndSerializeStudent,
    icon: const Icon(Icons.add),
    label: const Text('Add Student'),
```

```
      ),
    );
  }
}
```

[lib/student.dart]

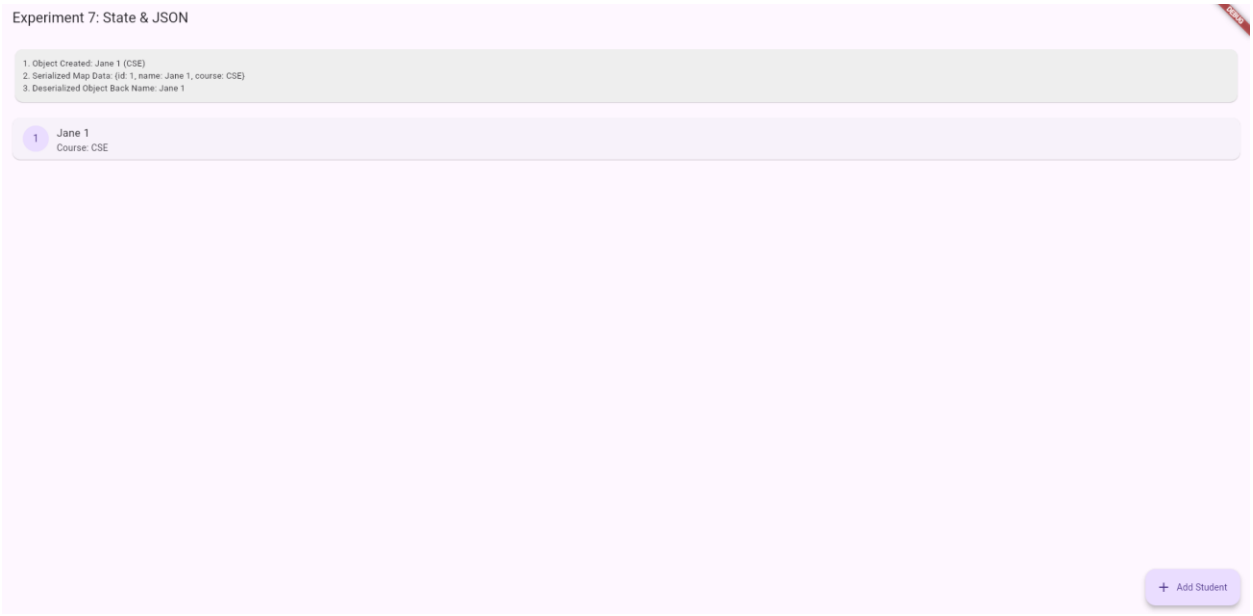
```
class Student {
  final int id;
  final String name;
  final String course;

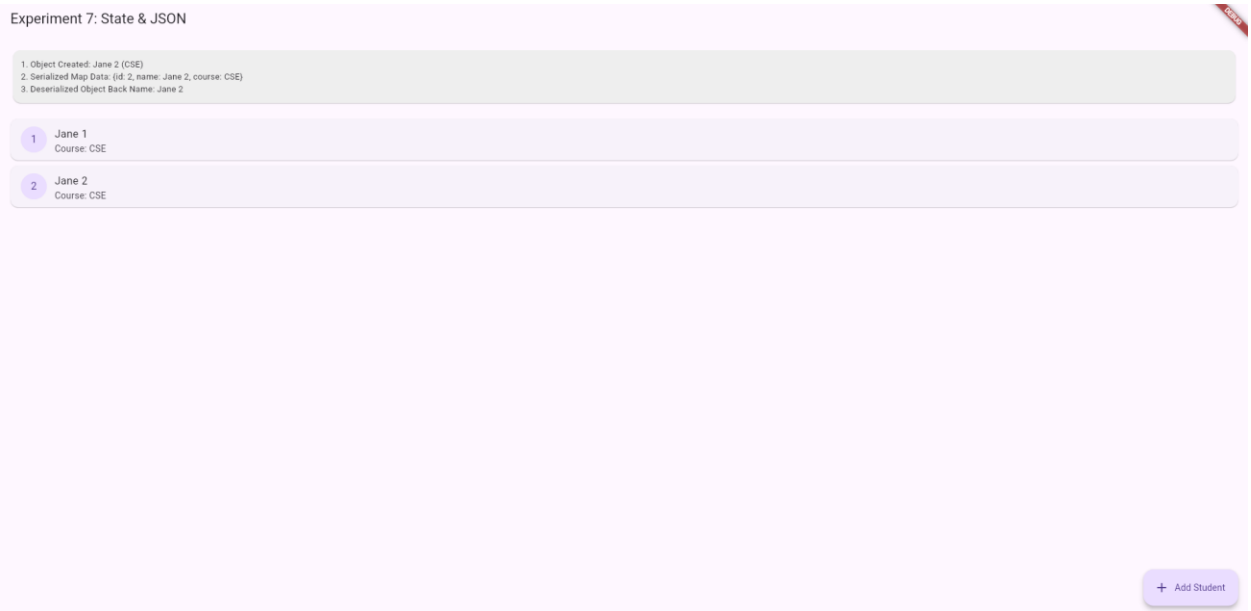
  Student({required this.id, required this.name, required this.course});

  // Convert Student object into a Map structure (Serialization)
  Map<String, dynamic> toJson() {
    return {'id': id, 'name': name, 'course': course};
  }
  // Parse a Map structure back into a Student object (Deserialization)
  factory Student.fromJson(Map<String, dynamic> json) {
    return Student(id: json['id'], name: json['name'], course: json['course']);
  }
}
```

Output


Experiment 7: State & JSON
Logs will appear here when you add a student.
No students added yet.
Add Student


Experiment 7: State & JSON
1. Object Created: Jane 1 (CSE)
2. Serialized Map Data: {id: 1, name: Jane 1, course: CSE}
3. Deserialized Object Back Name: Jane 1
1
Jane 1
Course: CSE
Add Student



Result: The Flutter application was implemented to manage changing student data using application state management. Student objects were converted into JSON-style map data and reconstructed using serialization and deserialization methods.

EX - 8	DATA PERSISTENCE USING SQLITE AND FIREBASE
DATE	12.10.26

Aim: To implement data persistence in a Flutter application using a local SQLite database and a cloud-based Firebase database.

Procedure

1. Create a new Flutter project named sqlite_app using the commands flutter create sqlite_app and cd sqlite_app.
2. Add the required SQLite packages, such as sqflite and path, to the pubspec.yaml file and execute flutter pub get.
3. Create a file named database_helper.dart inside the lib folder to manage database operations.
4. Create a SQLite database containing a students table with the fields id, name, and course.
5. Implement CRUD operations—Create, Read, Update, and Delete—to manage student records in the database.
6. Create TextFields to collect student names and course details, and insert the entered information into SQLite.
7. Retrieve student records from the database and display them dynamically using ListView.builder().
8. Test local data persistence by closing and restarting the application, then verify that previously stored student records remain available.
9. Create a project in the Firebase Console, configure Firebase for the Flutter application, add the required Firebase packages, and initialize Firebase before running the application.
10. Create a cloud collection named students with name and course fields. Implement operations to insert student information into Firebase and retrieve and display the stored records in the Flutter application.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

import 'firebase_options.dart';
import 'database_helper.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Student Management',
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),
      home: const StudentPage(),
    );
  }
}

class StudentPage extends StatefulWidget {
  const StudentPage({super.key});

  @override
  State<StudentPage> createState() => _StudentPageState();
}
```

```
class _StudentPageState extends State<StudentPage> {
  final nameController = TextEditingController();
  final courseController = TextEditingController();

  List<Map<String, dynamic>> students = [];
  String? editingId;
  bool loading = true;
  bool saving = false;

  @override
  void initState() {
    super.initState();
    loadStudents();
  }

  Future<void> loadStudents() async {
    try {
      final data = await DatabaseHelper.instance.getStudents();

      if (!mounted) return;

      setState(() {
```

```
      students = data;
      loading = false;
    });
  } catch (e) {
    if (!mounted) return;

    setState(() {
      loading = false;
    });

    showMessage('Database error: $e');
  }
}

void showMessage(String message) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(message)),
  );
}

Future<void> saveStudent() async {
  final name = nameController.text.trim();
```

```
  final course = courseController.text.trim();

  if (name.isEmpty || course.isEmpty) {
    showMessage('Please enter both name and course.');
    return;
  }

  if (saving) return;

  setState(() {
    saving = true;
  });

  try {
    final student = {
      'name': name,
      'course': course,
    };

    if (editingId == null) {
      await DatabaseHelper.instance.insertStudent(student);
    } else {
```

```
    await DatabaseHelper.instance.updateStudent(
      editingId!,
      student,
    );
  }

  if (!mounted) return;

  nameController.clear();
  courseController.clear();

  setState(() {
    editingId = null;
  });

  await loadStudents();

  if (!mounted) return;

  showMessage(
    'Student ${editingId == null ? 'saved' : 'updated'} successfully.',
  );
```

```
    } catch (e) {
      if (mounted) {
        showMessage('Unable to save student: $e');
      }
    } finally {
      if (mounted) {
        setState(() {
          saving = false;
        });
      }
    }
  }

  void editStudent(Map<String, dynamic> student) {
    nameController.text = student['name'].toString();
    courseController.text = student['course'].toString();

    setState(() {
      editingId = student['id'].toString();
    });
  }
```

```
 void cancelEdit() {
  nameController.clear();
  courseController.clear();

  setState(() {
   editingId = null;
  });
 }

 Future<void> removeStudent(String id) async {
  try {
   await DatabaseHelper.instance.deleteStudent(id);
   await loadStudents();

   if (mounted) {
    showMessage('Student deleted successfully.');
   }
  } catch (e) {
   if (mounted) {
    showMessage('Unable to delete student: $e');
   }
  }
```

```
 }

 @override
 void dispose() {
  nameController.dispose();
  courseController.dispose();
  super.dispose();
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: const Text('Student Management'),
    centerTitle: true,
   ),
   body: Center(
    child: ConstrainedBox(
     constraints: const BoxConstraints(maxWidth: 650),
     child: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(
```

```
children: [
  TextField(
    controller: nameController,
    decoration: const InputDecoration(
      labelText: 'Student Name',
      border: OutlineInputBorder(),
    ),
  ),
  const SizedBox(height: 12),
  TextField(
    controller: courseController,
    decoration: const InputDecoration(
      labelText: 'Course',
      border: OutlineInputBorder(),
    ),
  ),
  const SizedBox(height: 12),
  SizedBox(
    width: double.infinity,
    child: ElevatedButton(
      onPressed: saving ? null : saveStudent,
      child: Text(
```

```
        saving
            ? 'Please wait...'
            : editingId == null
                ? 'Add Student'
                : 'Update Student',
      ),
    ),
  ),
  if (editingId != null)
    TextButton(
      onPressed: cancelEdit,
      child: const Text('Cancel Edit'),
    ),
  const SizedBox(height: 20),
  const Align(
    alignment: Alignment.centerLeft,
    child: Text(
      'Student List',
      style: TextStyle(
        fontSize: 20,
        fontWeight: FontWeight.bold,
      ),
```

```
  ),
),
const SizedBox(height: 8),
Expanded(
  child: loading
      ? const Center(
          child: CircularProgressIndicator(),
        )
      : students.isEmpty
          ? const Center(
              child: Text('No students added yet.'),
            )
          : ListView.builder(
              itemCount: students.length,
              itemBuilder: (context, index) {
                final student = students[index];


                return Card(
                  child: ListTile(
                    title: Text(
                      student['name'].toString(),
                    ),
```

```
subtitle: Text(
  student['course'].toString(),
),
trailing: Wrap(
  spacing: 4,
  children: [
    IconButton(
      tooltip: 'Edit student',
      icon: const Icon(Icons.edit),
      onPressed: () {
        editStudent(student);
      },
    ),
    IconButton(
      tooltip: 'Delete student',
      icon: const Icon(
        Icons.delete,
        color: Colors.red,
      ),
      onPressed: () {
        removeStudent(
          student['id'].toString(),
```

```
                                    );
                                  },
                                ),
                              ],
                            ),
                          ),
                        );
                      },
                    ),
                  ),
                ],
              ),
            ),
          ),
        ),
      );
  }
}
```

[lib/database_helper.dart]

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

```
class DatabaseHelper {
  DatabaseHelper._init();

  static final DatabaseHelper instance = DatabaseHelper._init();

  final CollectionReference<Map<String, dynamic>> _students =
      FirebaseFirestore.instance.collection('students');

  Future<List<Map<String, dynamic>>> getStudents() async {
    final snapshot = await _students
        .orderBy('createdAt', descending: true)
        .get();

    return snapshot.docs.map((doc) {
      return {
        ...doc.data(),
        'id': doc.id,
      };
    }).toList();
  }
```

```
Future<String> insertStudent(
  Map<String, dynamic> student,
) async {
  final doc = await _students.add({
    'name': student['name'],
    'course': student['course'],
    'createdAt': FieldValue.serverTimestamp(),
  });

  return doc.id;
}

Future<void> updateStudent(
  String id,
  Map<String, dynamic> student,
) async {
  await _students.doc(id).update({
    'name': student['name'],
    'course': student['course'],
  });
}
```

```
  Future<void> deleteStudent(String id) async {
    await _students.doc(id).delete();
  }
}
```

[lib/firebase_helper.dart]

```
import 'package:cloud_firestore/cloud_firestore.dart';

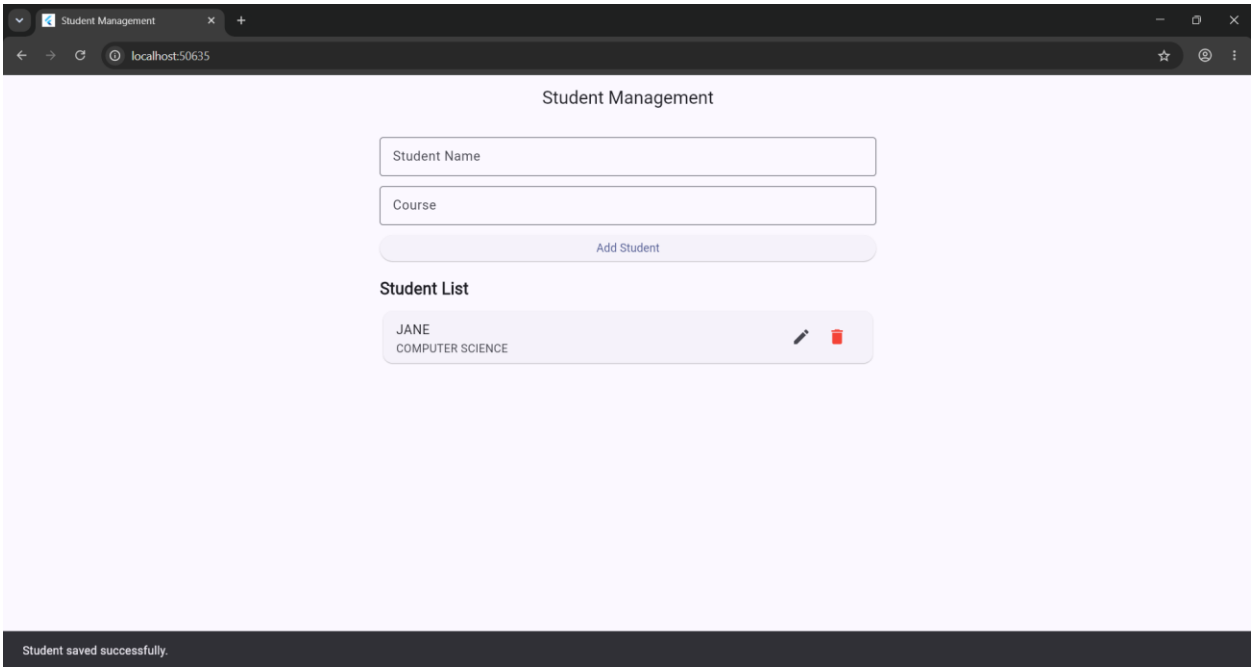
class FirebaseHelper {
  final CollectionReference<Map<String, dynamic>> students =
      FirebaseFirestore.instance.collection('students');

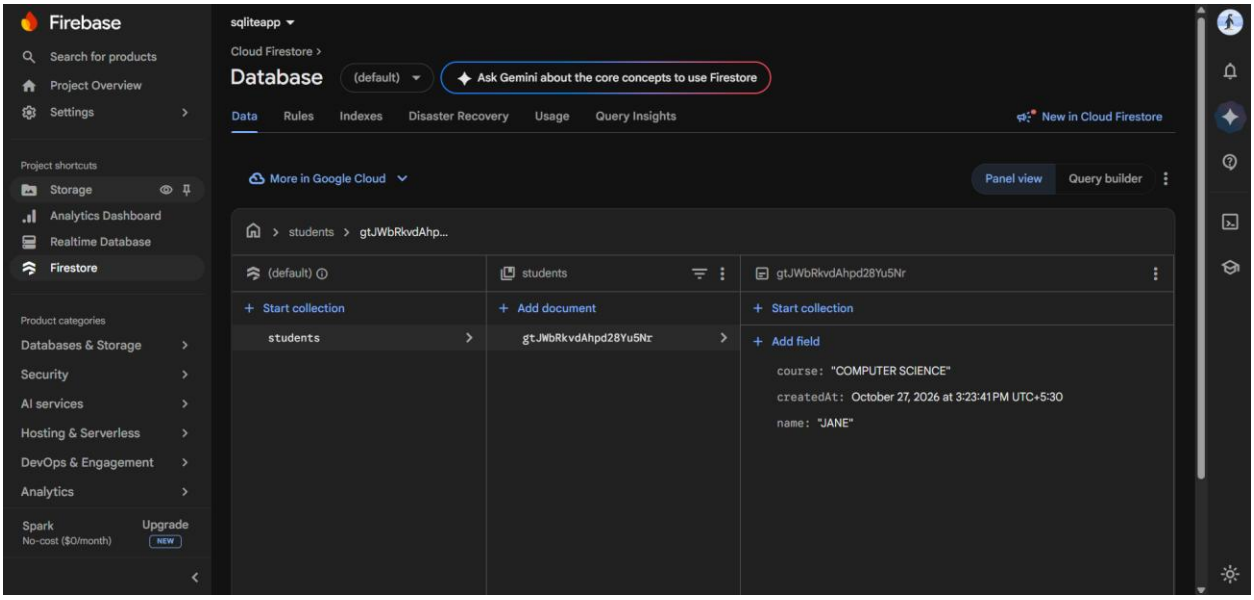
  Future<void> addStudent(String name, String course) async {
    await students.add({
      'name': name,
      'course': course,
      'createdAt': FieldValue.serverTimestamp(),
    });
  }

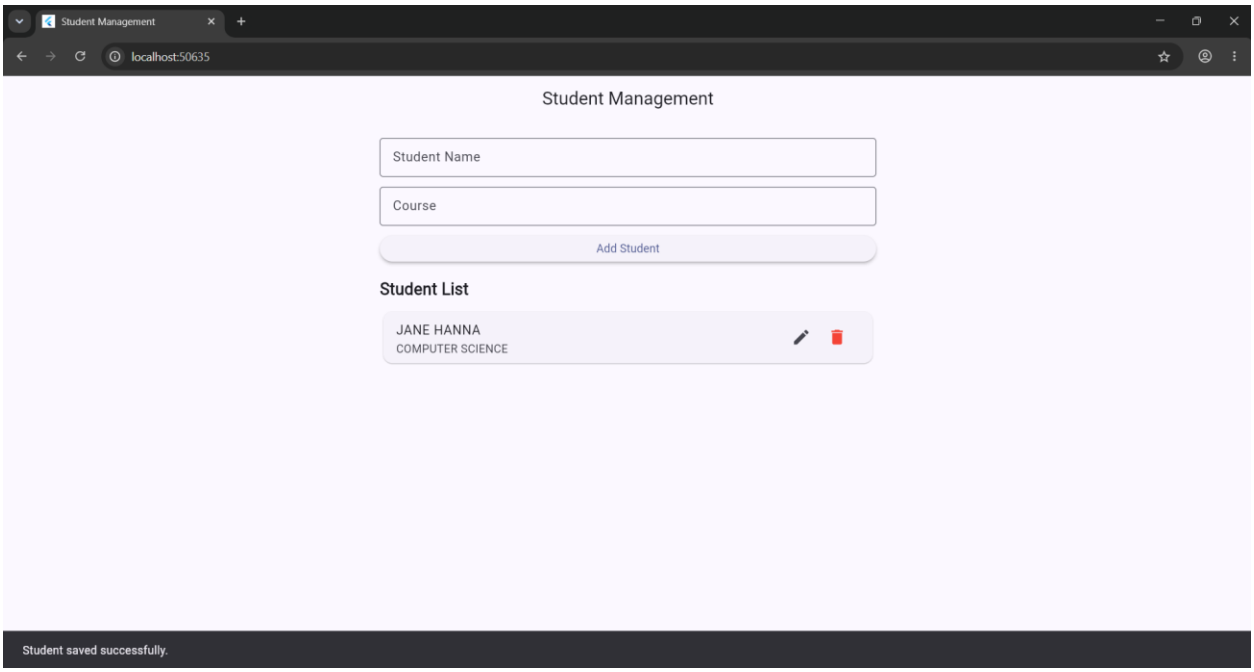
  Stream<QuerySnapshot<Map<String, dynamic>>> getStudents() {
```

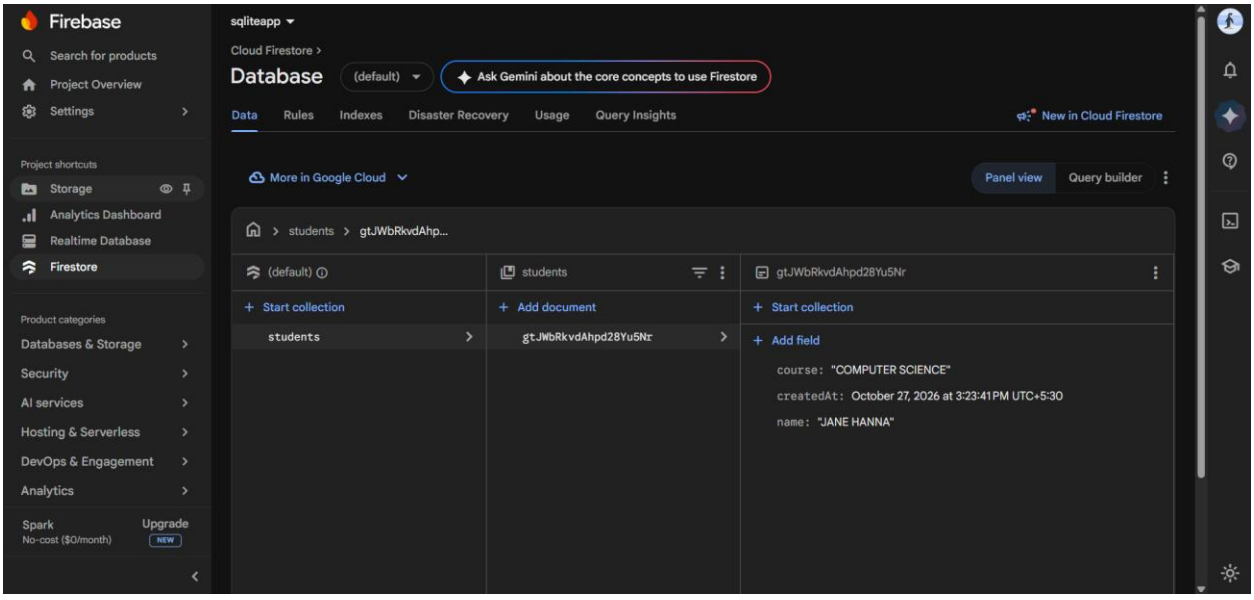
```
    return students.orderBy('createdAt', descending: true).snapshots();
  }
}
```

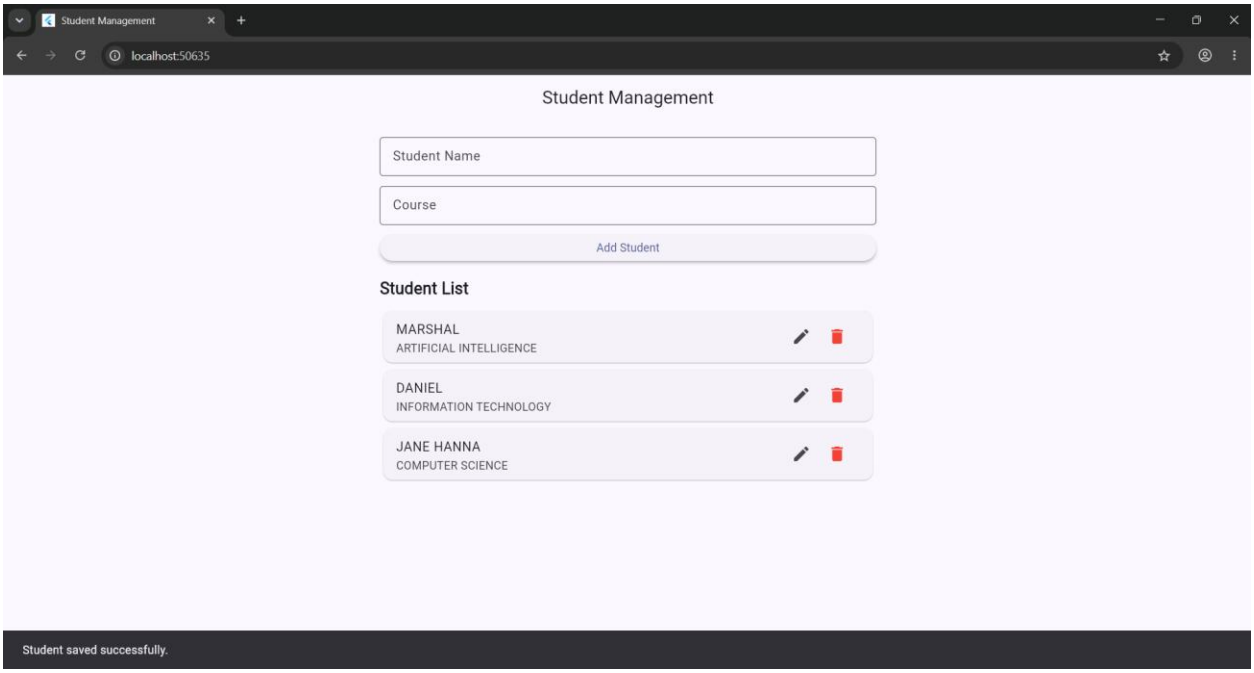
Output

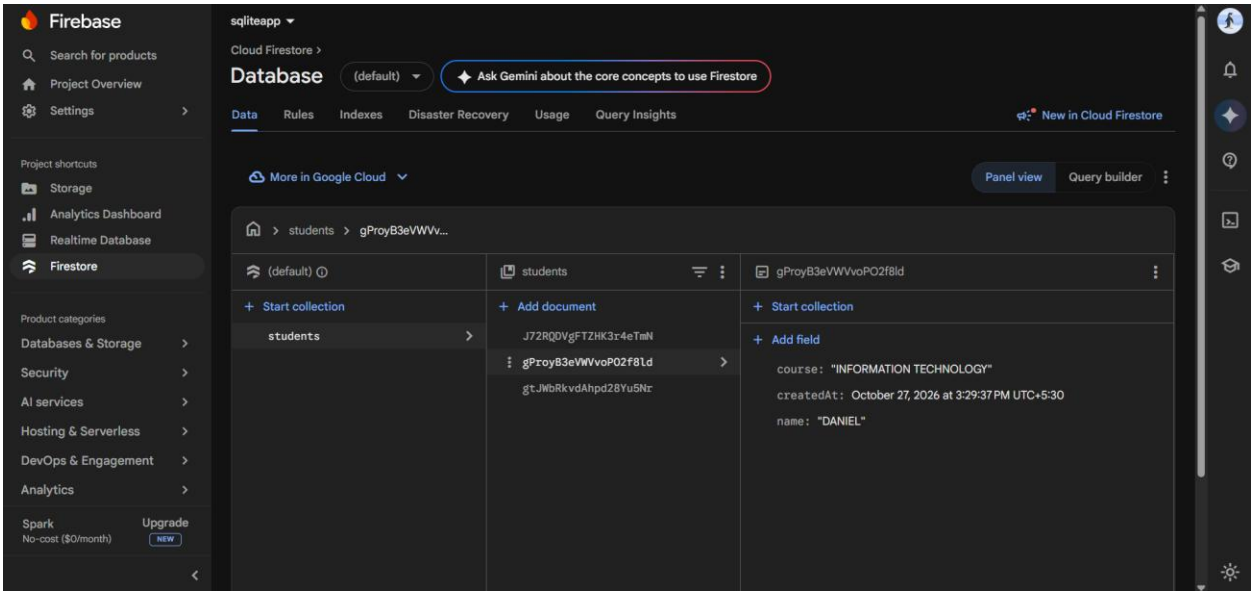


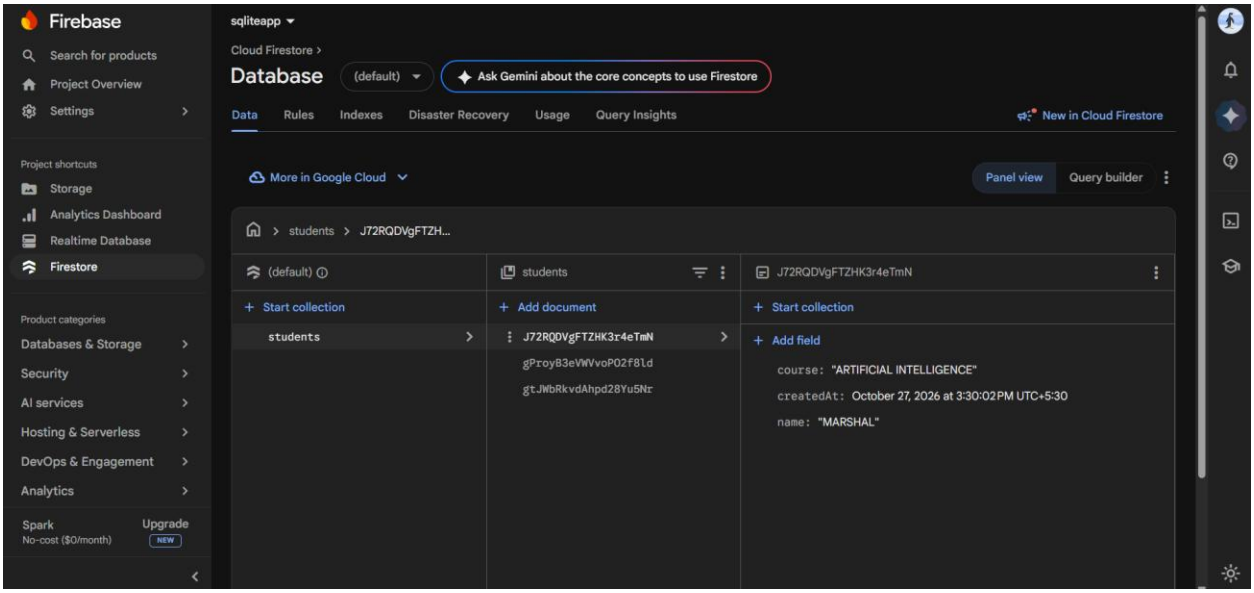


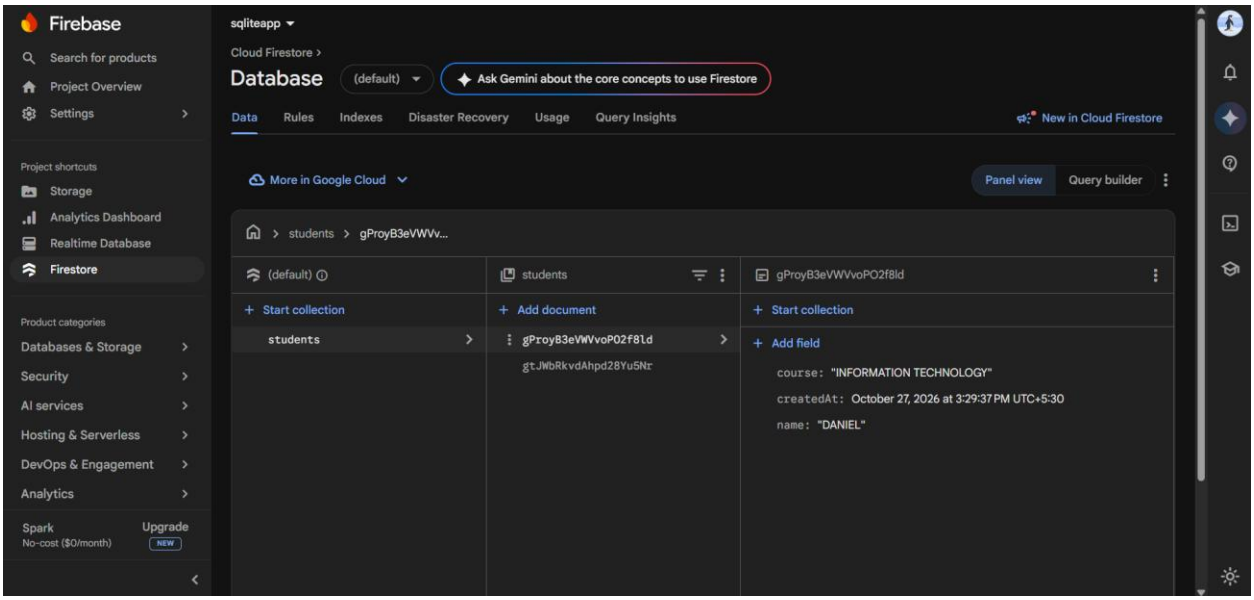

Student Management
localhost:50635
Student Management
Student Name
Course
Add Student
Student List
JANE HANNA
COMPUTER SCIENCE
Student saved successfully.


Firebase
Search for products
Project Overview
Settings
Project shortcuts
Storage
Analytics Dashboard
Realtime Database
Firestore
Product categories
Databases & Storage
Security
AI services
Hosting & Serverless
DevOps & Engagement
Analytics
Spark
No-cost ($0/month)
Upgrade
sqliteapp
Cloud Firestore
Database
(default)
Ask Gemini about the core concepts to use Firestore
Data
Rules
Indexes
Disaster Recovery
Usage
Query Insights
New in Cloud Firestore
More in Google Cloud
Panel view
Query builder
students > gtJWbRkvdAhp...
(default)
students
gtJWbRkvdAhpd28Yu5Nr
Start collection
Add document
Start collection
Add field
students
gtJWbRkvdAhpd28Yu5Nr
course: "COMPUTER SCIENCE"
createdAt: October 27, 2026 at 3:23:41PM UTC+5:30
name: "JANE HANNA"


Student Management
localhost:50635
Student Management
Student Name
Course
Add Student
Student List
MARSHAL
ARTIFICIAL INTELLIGENCE
DANIEL
INFORMATION TECHNOLOGY
JANE HANNA
COMPUTER SCIENCE
Student saved successfully.


Firebase
Search for products
Project Overview
Settings
Project shortcuts
Storage
Analytics Dashboard
Realtime Database
Firestore
Product categories
Databases & Storage
Security
AI services
Hosting & Serverless
DevOps & Engagement
Analytics
Spark
No-cost ($0/month)
Upgrade
sqliteapp
Cloud Firestore
Database
(default)
Ask Gemini about the core concepts to use Firestore
Data
Rules
Indexes
Disaster Recovery
Usage
Query Insights
New in Cloud Firestore
More in Google Cloud
Panel view
Query builder
students > gProyB3eVWVv...
(default)
students
gProyB3eVWVvoPO2f8ld
Start collection
students
Add document
J72RQDVgFTZHK3r4eTmN
gProyB3eVWVvoPO2f8ld
gtJWbRkvdAhpd28Yu5Nr
Start collection
Add field
course: "INFORMATION TECHNOLOGY"
createdAt: October 27, 2026 at 3:29:37 PM UTC+5:30
name: "DANIEL"





Result: The Flutter application was developed to demonstrate data persistence using SQLite for local storage and Firebase for cloud storage.

EX - 9	ACCESSING APIs THROUGH HTTP
DATE	12.10.26

Aim: To access a REST API using HTTP, retrieve JSON data, and display the data in a Flutter application.

Procedure

1. Create a new Flutter project named api_app using VS Code and open the project folder.
2. Add the http package to the project by executing flutter pub add http in the terminal.
3. Import the required libraries, dart:convert, flutter/material.dart, and http, into the main.dart file.
4. Create the Flutter application interface using MaterialApp, Scaffold, and AppBar widgets.
5. Define an asynchronous function named fetchData() to send an HTTP GET request to the DummyJSON Posts API using http.get().
6. Check the HTTP response status code. If the status code is 200, decode the JSON response using jsonDecode() and extract the posts from the posts field.
7. Store the retrieved post records in a list and update the application state using setState().
8. Display the retrieved post IDs, titles, and descriptions using ListView.builder() and ListTile widgets.
9. Display a loading indicator while the API request is processing and show an error message with a retry option if the request fails.
10. Run the application in Chrome using flutter run -d chrome. Verify that the posts are displayed correctly and test the refresh button to retrieve the data again.

Program

[lib/main.dart]

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'API Application',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
```

```
      ),
      home: const ApiPage(),
    );
  }
}

class ApiPage extends StatefulWidget {
  const ApiPage({super.key});

  @override
  State<ApiPage> createState() => _ApiPageState();
}

class _ApiPageState extends State<ApiPage> {
  List<dynamic> posts = [];
  bool isLoading = true;
  String? errorMessage;

  @override
  void initState() {
    super.initState();
    fetchData();
```

```
  }

  Future<void> fetchData() async {
    setState(() {
      isLoading = true;
      errorMessage = null;
    });

    try {
      final response = await http
          .get(
            Uri.parse('https://dummyjson.com/posts?limit=30'),
          )
          .timeout(const Duration(seconds: 20));

      if (response.statusCode == 200) {
        final Map<String, dynamic> decodedData =
            jsonDecode(response.body) as Map<String, dynamic>;

        final List<dynamic> fetchedPosts =
            decodedData['posts'] as List<dynamic>;
```

```
      if (!mounted) return;


      setState(() {
        posts = fetchedPosts;
        isLoading = false;
      });
    } else {
      throw Exception(
        'Request failed with status ${response.statusCode}',
      );
    }
  } catch (e) {
    if (!mounted) return;


    setState(() {
      errorMessage =
          'Unable to load posts. Please check your internet connection.';
      isLoading = false;
    });
  }
}
```

```
 @override
 Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
     title: const Text('REST API - Posts'),
     centerTitle: true,
     actions: [
      IconButton(
       onPressed: isLoading ? null : fetchData,
       icon: const Icon(Icons.refresh),
       tooltip: 'Refresh posts',
      ),
     ],
    ),
    body: isLoading
       ? const Center(
          child: CircularProgressIndicator(),
         )
       : errorMessage != null
          ? Center(
             child: Padding(
               padding: const EdgeInsets.all(20),
```

```
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const Icon(
                Icons.error_outline,
                size: 48,
                color: Colors.red,
              ),
              const SizedBox(height: 12),
              Text(
                errorMessage!,
                textAlign: TextAlign.center,
              ),
              const SizedBox(height: 12),
              ElevatedButton(
                onPressed: fetchData,
                child: const Text('Try Again'),
              ),
            ],
          ),
        ),
      )
```

```
: Column(
    children: [
      Padding(
        padding: const EdgeInsets.all(16),
        child: Text(
          'Posts Retrieved: ${posts.length}',
          style: const TextStyle(
            fontSize: 20,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
      Expanded(
        child: posts.isEmpty
            ? const Center(
                child: Text('No posts available.'),
              )
            : ListView.builder(
                itemCount: posts.length,
                itemBuilder: (context, index) {
                  final post =
                      posts[index] as Map<String, dynamic>;
```

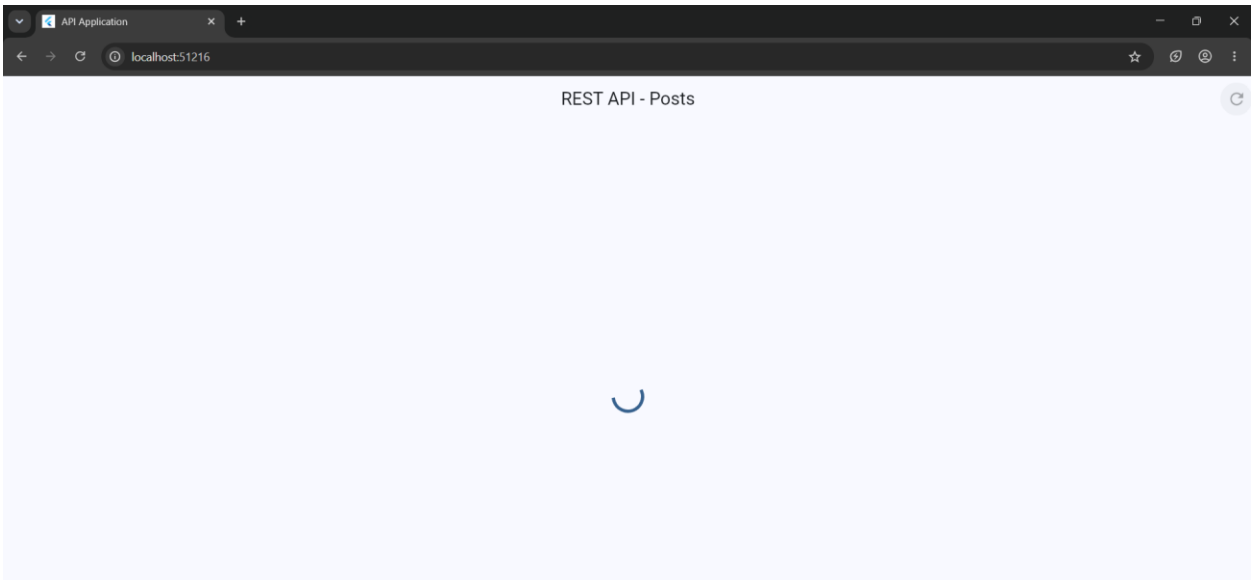
```
final title =
    post['title']?.toString() ??
        'No title';

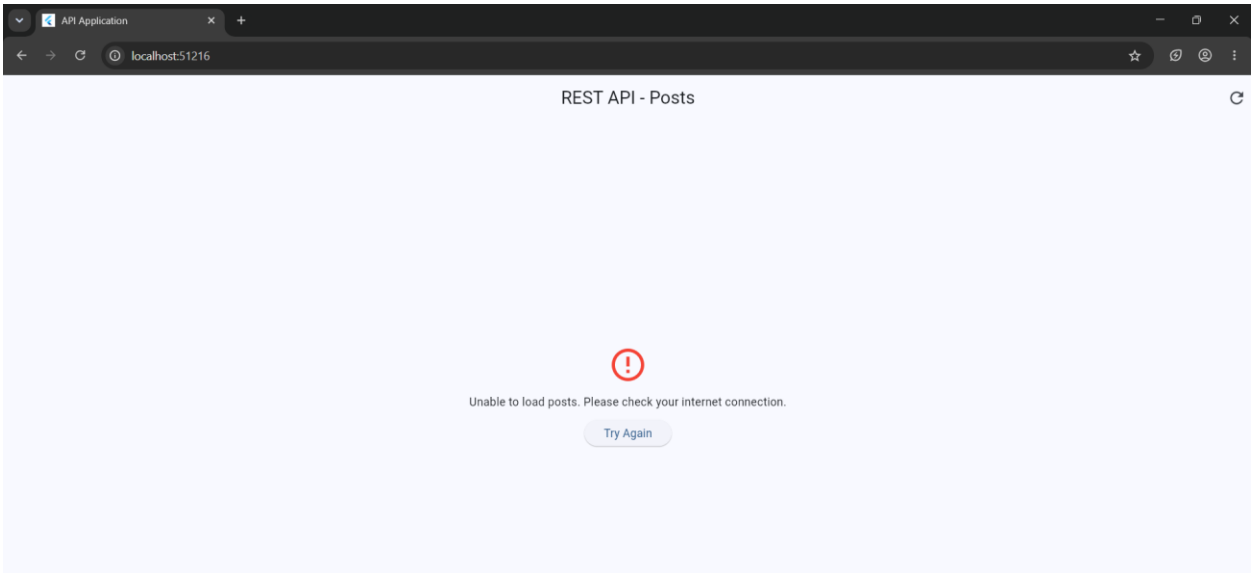
final body =
    post['body']?.toString() ??
        'No description';

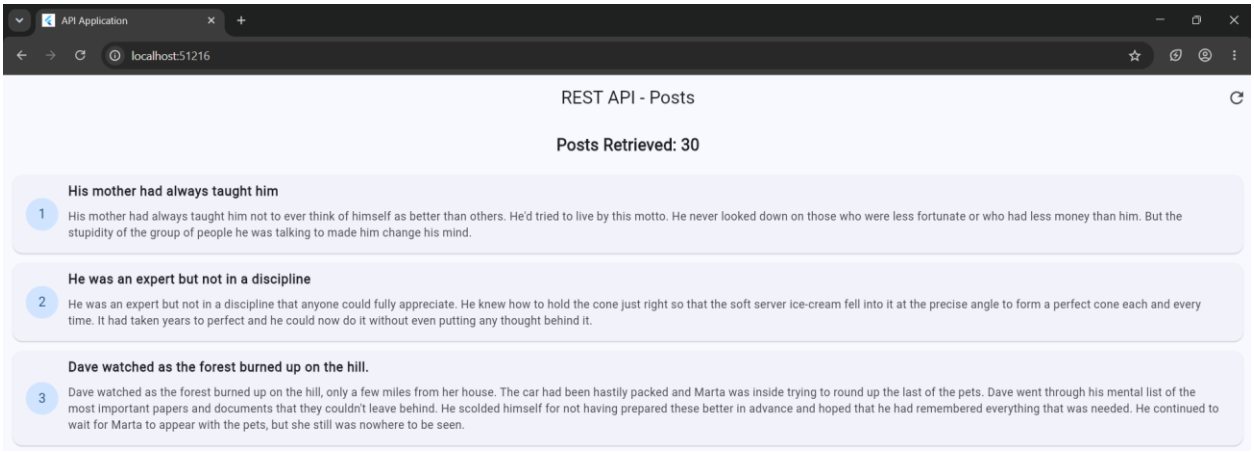
return Card(
  margin: const EdgeInsets.symmetric(
    horizontal: 12,
    vertical: 6,
  ),
  child: ListTile(
    leading: CircleAvatar(
      child: Text(
        '${post['id']}',
      ),
    ),
    title: Text(
      title,
```

```
                    style: const TextStyle(
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  subtitle: Padding(
                    padding: const EdgeInsets.only(
                      top: 8,
                      bottom: 8,
                    ),
                    child: Text(body),
                  ),
                ),
              );
            },
          ),
        ),
      ],
    ),
  );
 }
}
```

Output


API Application
localhost:51216
REST API - Posts


API Application
localhost:51216
REST API - Posts
Unable to load posts. Please check your internet connection.
Try Again



Result: The Flutter application successfully demonstrates REST API access using the HTTP package.

EX - 10	ADD ANDROID AS A BUILD TARGET
DATE	02.11.26

Aim: To configure Flutter for Android development, run a Flutter application on an Android emulator, and generate an APK file for installation.

Procedure

1. Check Flutter installation: Open the terminal in VS Code and execute flutter doctor to verify the Flutter environment and Android development setup.
2. Check available devices: Execute flutter devices to identify the connected Android emulator or physical device.
3. Configure Android licenses: Run flutter doctor --android-licenses and accept the required Android SDK licenses.
4. Create a Flutter project: Execute flutter create android_app to create a new Flutter application with Android build support.
5. Open the project: Navigate to the project directory using cd android_app and open it in VS Code.
6. Start the Android emulator: Launch an Android virtual device through Android Studio's Device Manager and wait for it to start.
7. Run the application: Execute flutter run -d emulator-5554 to build, install, and launch the Flutter application on the Android emulator.
8. Build the APK file: Execute flutter build apk --release to generate a release APK for the application.
9. Install the APK: Locate the generated file at build/app/outputs/flutter-apk/app-release.apk and install it on the emulator using the adb install command.

10. Test the application: Open the installed application in the emulator and verify that it launches and functions as expected.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';


void main() {
  runApp(const MyApp());
}


class MyApp extends StatelessWidget {
  const MyApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: .fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
```

```
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

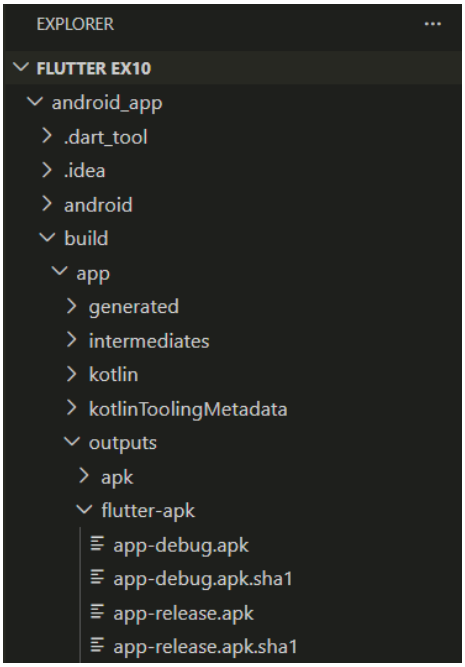
  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

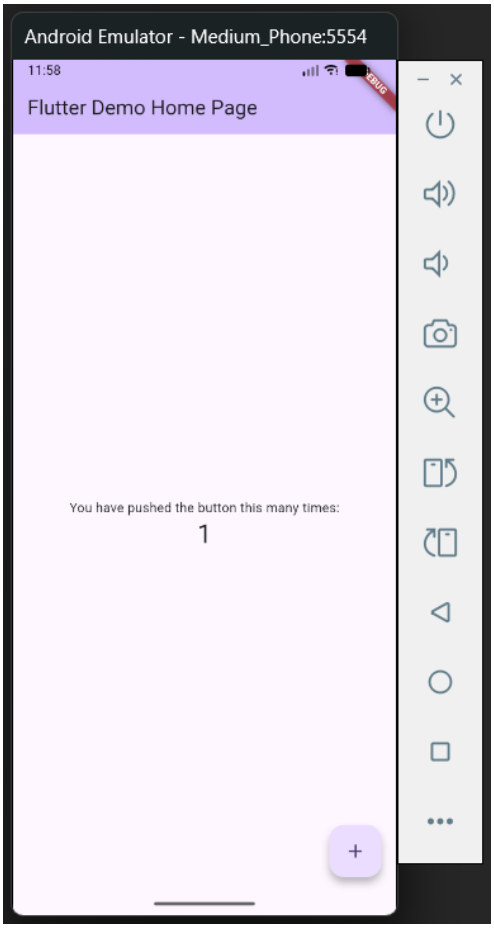
  @override
```

```
Widget build(BuildContext context) {
 return Scaffold(
  appBar: AppBar(
   backgroundColor: Theme.of(context).colorScheme.inversePrimary,
   title: Text(widget.title),
  ),
  body: Center(
   child: Column(
    mainAxisAlignment: .center,
    children: [
     const Text('You have pushed the button this many times:'),
     Text(
      '$_counter',
      style: Theme.of(context).textTheme.headlineMedium,
     ),
    ],
   ),
  ),
  floatingActionButton: FloatingActionButton(
   onPressed: _incrementCounter,
   tooltip: 'Increment',
   child: const Icon(Icons.add),
```

```
    ),
  );
 }
}
```

Output





Result: The Flutter application was successfully built, installed, and tested on the Android emulator.

EX - 11	ADD iOS AS A BUILD TARGET
DATE	02.11.26

Aim: To add iOS as a build target for a Flutter application, run it on an iOS Simulator using Xcode, and test its UI, navigation, text fields, buttons, images and screen transitions.

Procedure

1. Install the requirements: Ensure that macOS, Xcode, Flutter SDK, and CocoaPods are installed and configured.
2. Check Flutter installation: Execute flutter doctor to verify the Flutter development environment.
3. Check available devices: Run flutter devices to identify the available iOS simulators and connected devices.
4. Create a Flutter project: Execute flutter create ios_app to create a new Flutter application.
5. Navigate to the project: Run cd ios_app to enter the newly created project directory.
6. Open the iOS project in Xcode: Execute open ios/Runner.xcworkspace to open the project in Xcode.
7. Select an iOS Simulator: Choose an available simulator, such as iPhone 15, in Xcode.
8. Run the application: Execute flutter run in the terminal or press the Run button in Xcode to launch the application.
9. Test the application: Verify the UI, navigation, text fields, buttons, images, and screen transitions.
10. Build the iOS application: Execute flutter build ios --simulator for a simulator build or flutter build ipa for distribution (requires a signing team configured in Xcode).

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';


void main() {
  runApp(const MyApp());
}


class MyApp extends StatelessWidget {
  const MyApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'iOS App',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
      ),
      home: const HomePage(),
    );
  }
```

```
}

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  final controller = TextEditingController();

  @override
  void dispose() {
    controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
```

```
  title: const Text('iOS Build Target'),
),
body: Padding(
  padding: const EdgeInsets.all(20),
  child: Column(
    children: [
      const FlutterLogo(size: 90),
      const SizedBox(height: 20),

      TextField(
        controller: controller,
        decoration: const InputDecoration(
          labelText: 'Enter your name',
          border: OutlineInputBorder(),
        ),
      ),

      const SizedBox(height: 20),

      ElevatedButton(
        onPressed: () {
          Navigator.push(
```

```
                context,
                MaterialPageRoute(
                  builder: (_) => SecondPage(
                    name: controller.text,
                  ),
                ),
              );
            },
            child: const Text('Go to Next Screen'),
          ),
        ],
      ),
    ),
  );
 }
}


class SecondPage extends StatelessWidget {
  final String name;

  const SecondPage({
    super.key,
```

```
    required this.name,
  });


  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Screen'),
      ),
      body: Center(
        child: Text(
          'Hello, $name! Running on iOS',
          style: const TextStyle(fontSize: 22),
        ),
      ),
    );
  }
}
```

Output



The Flutter application was configured with iOS as a build target, run on the iOS Simulator and tested successfully for UI, navigation, text fields, buttons, images and screen transitions (requires a macOS + Xcode environment).



Result: The Flutter application was configured, tested, and built for the iOS target using the required macOS and Xcode environment.

EX - 12	BUILD WEB APP WITH FLUTTER
DATE	02.11.26

Aim: To develop, execute, and build a responsive Flutter application for the Web using Flutter SDK and Google Chrome.

Procedure

1. Check Flutter Web support: Execute flutter doctor to verify the Flutter development environment.
2. Check available devices: Run flutter devices to identify Chrome or other configured web browsers.
3. Enable web support: Execute flutter config --enable-web if web support is not already enabled.
4. Create a Flutter project: Run flutter create flutter_web_app to create a new Flutter application.
5. Open the project: Navigate to the project directory using cd flutter_web_app and open it in VS Code using code ..
6. Run the application: Execute flutter run -d chrome to launch the Flutter application in Google Chrome.
7. Create the web interface: Implement a Scaffold with an AppBar, Flutter logo, Welcome button, and suitable text.
8. Make the application responsive: Use LayoutBuilder to display Mobile Web Layout for screen widths below 600 pixels and Desktop Web Layout for larger screens.
9. Test the application: Resize the Chrome browser window and verify the responsive layout and button functionality.

10. Build the web application: Execute flutter build web and verify that the production files are generated inside the build/web/ directory.

Program

[lib/main.dart]

```
import 'package:flutter/material.dart';


void main() {
  runApp(const MyApp());
}


class MyApp extends StatelessWidget {
  const MyApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Web Application',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
```

```
      home: const HomePage(),
    );
  }
}

class HomePage extends StatelessWidget {
  const HomePage({super.key});

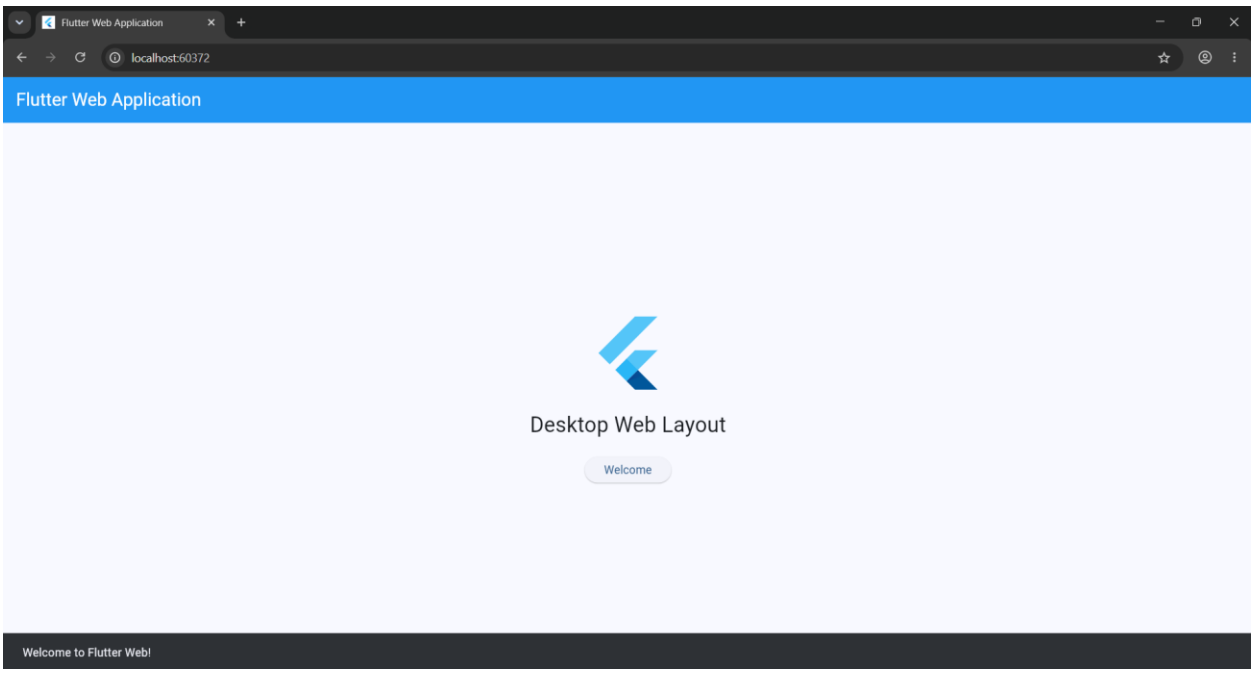
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Flutter Web Application'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ),
      body: LayoutBuilder(
        builder: (context, constraints) {
          if (constraints.maxWidth < 600) {
            return const Center(
              child: Text(
                'Mobile Web Layout',
```

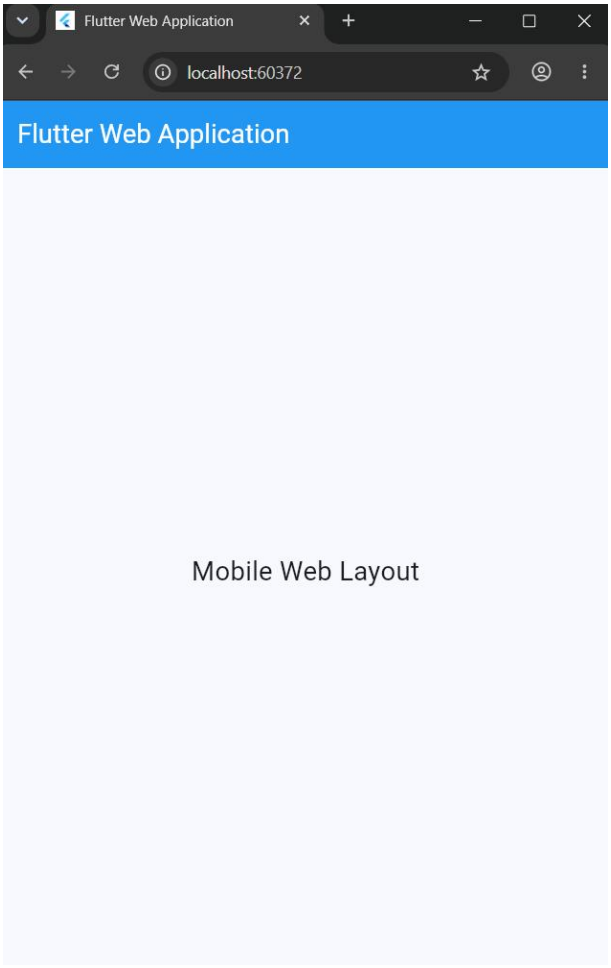
```
      style: TextStyle(fontSize: 22),
    ),
  );
}

return Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      const FlutterLogo(size: 100),
      const SizedBox(height: 20),
      const Text(
        'Desktop Web Layout',
        style: TextStyle(fontSize: 26),
      ),
      const SizedBox(height: 20),
      ElevatedButton(
        onPressed: () {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
              content: Text('Welcome to Flutter Web!'),
            ),
```

```
                );
              },
              child: const Text('Welcome'),
            ),
          ],
        ),
      );
    },
  ),
 );
 }
}
```

Output







Result: The responsive Flutter web application was successfully developed, executed, tested, and built for the Web.